作业 _Week9

20244227004 蔡博凯

2024 年 11 月 5 日

第一题

试用EM算法估计三硬币模型的参数（李航例9.1），初值取为

$$\pi^{(0)}=0.46,\ p^{(0)}=0.55,\ q^{(0)}=0.67$$

例 9.1（三硬币模型） 假设有 3 枚硬币，分别记作 A，B，C。这些硬币正面出现的概率分别是 π，p 和 q。进行如下掷硬币试验：先掷硬币 A，根据其结果选出硬币 B 或硬币 C，正面选硬币 B，反面选硬币 C；然后掷选出的硬币，掷硬币的结果，出现正面记作 1，出现反面记作 0；独立地重复 n 次试验（这里，$n=10$），观测结果如下：

$$1,1,0,1,0,0,1,0,1,1$$

假设只能观测到掷硬币的结果，不能观测掷硬币的过程。问如何估计三硬币正面出现的概率，即三硬币模型的参数。

解答

解答思路

1. 列出例 9.1 的三硬币模型；
2. 写出三硬币模型的 EM 算法；
3. 根据上述 EM 算法，编写代码，并求出模型参数的极大似然估计。

解答步骤

第 1 步：例 9.1 的三硬币模型

例 9.1（三硬币模型）假设有 3 枚硬币，分别记作 A、B、C，这些硬币正面出现的概率分别是 π，p 和 q。进行如下掷硬币试验：先掷硬币 A，根据其结果选择硬币 B 或硬币 C，正面选硬币 B，反面选硬币 C；然后掷选出的硬币，掷硬币的结果，出现正面记作 1，出现反面记作 0。独立地重复 n 次试验（这里，$n=10$），观测结果如下：

$$1,1,0,1,0,0,1,0,1,1$$

假设只能观测到掷硬币的结果，不能观测到掷硬币的过程。三硬币模型可以写作

$$\begin{aligned} P(y|\theta) &= \sum_z P(y,z|\theta) = \sum_z P(z|\theta)P(y|z,\theta) \\ &= \pi p^y(1-p)^{1-y} + (1-\pi)q^y(1-q)^{1-y} \end{aligned}$$

这里：

1. 随机变量 y 是观测变量，表示一次试验观测的结果是 1 或 0；
2. 随机变量 z 是隐藏变量，表示未观测到的掷硬币的结果；
3. $\theta=(\pi,p,q)$ 是模型的参数。

第 2 步：三硬币模型的 EM 算法

EM 算法首先选取参数的初值，记作 $\theta^{(0)}=(\pi^{(0)},p^{(0)},q^{(0)})$，然后通过下面的步骤迭代计算参数的估计值，直至收敛为止。第 i 次迭代的估计值为 $\theta^{(i)}=(\pi^{(i)},p^{(i)},q^{(i)})$。EM 算法的第 $i+1$ 次迭代如下：

E 步：计算在模型参数 $(\pi^{(i)},p^{(i)},q^{(i)})$ 下观测数据 y_j 来自抛硬币 B 的概率

$$\mu_j^{(i+1)} = \frac{\pi^{(i)}(p^{(i)})^{y_j}(1-p^{(i)})^{1-y_j}}{\pi^{(i)}(p^{(i)})^{y_j}(1-p^{(i)})^{1-y_j} + (1-\pi^{(i)})(q^{(i)})^{y_j}(1-q^{(i)})^{1-y_j}}$$

M 步：计算模型参数的新的估计值

$$\pi^{(i+1)} = \frac{1}{n}\sum_{j=1}^{N}\mu_j^{(i+1)}$$

$$p^{(i+1)} = \frac{\sum_{j=1}^{n}\mu_j^{(i+1)}y_j}{\sum_{j=1}^{n}\mu_j^{(i+1)}}$$

$$q^{(i+1)} = \frac{\sum_{j=1}^{n}(1-\mu_j^{(i+1)})y_j}{\sum_{j=1}^{n}(1-\mu_j^{(i+1)})}$$

第 3 步：编写代码并求出模型参数的极大似然估计 代码验证

可在附录查看完整代码，请参考 三硬币模型。

计算初始化参数为 (0.46,0.55,0.67) 的情况如下

```
模型参数的初值：
prob_A=0.46, prob_B=0.55, prob_C=0.67
EM算法训练过程：
第1次: prob_A=0.4619, prob_B=0.5346, prob_C=0.6561
第2次: prob_A=0.4619, prob_B=0.5346, prob_C=0.6561
模型参数的极大似然估计：
prob_A=0.4619, prob_B=0.5346, prob_C=0.6561
```

图 1: 运行结果

可见通过两次迭代,模型参数已经收敛,三硬币正面出现的概率分别为0.4619，0.5346，0.6561

第二题

试根据图2用吉布斯采样法推断以下概率

$P(\text{过劳死} \mid \text{学校政策} = \text{true}, \text{工作压力} = \text{true})$

仅需迭代一轮T=1, 初值赋为$\pi^0 = (C, W, U, \sim B, \sim D)$,
并请按照$C \to B \to D$顺序采样
(随机数假定为 $a_C = 0.13$, $a_B = 0.04$, $a_D = 0.77$)。

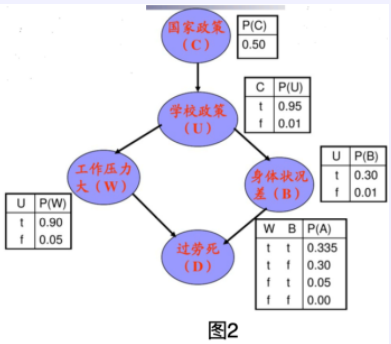


图2

假设最终采样T=1000轮，其中下列情况出现的次数分别为
$n(D, C, U, W, B) = 25$, $n(D, \sim C, U, W, B) = 20$, $n(D, C, U, W, \sim B) = 12$, $n(D, \sim C, U, W, \sim B) = 7$
请问上述概率的近似值。

解答

整理问题

使用 X 表示事件为 true, $\sim X$ 表示事件为 false.

1. 进行一次顺序采样

2. 求解 $P(D|U,W)$

解题步骤

第 1 问. 进行一次顺序采样

给定初值:

$$\pi_0 = (C, W, U, \sim B, \sim D)$$

1. 采样 C:

$$\begin{aligned} P(C|U) &= \frac{P(C,U)}{P(U)} \\ &= \frac{P(C)P(U|C)}{P(C)P(U|C) + P(\sim C)P(U|\sim C)} \\ &= \frac{0.5 \times 0.95}{0.5 \times 0.95 + 0.5 \times 0.01} \\ &\approx 0.9896 \end{aligned}$$

由于 $a_C = 0.13 < P(C|U)$ 故转移到下一个状态

$$\pi_1 = (\sim C, W, U, \sim B, \sim D)$$

2. 采样 B:

$$P(B|W,U,\sim D) = \frac{P(B,W,U,\sim D)}{P(W,U,\sim D)}$$

$$P(B,W,U,\sim D) = P(U) \cdot P(W|U) \cdot P(B|U) \cdot P(\sim D|W,B)$$

$$P(W,U,\sim D) = P(B,W,U,\sim D) + P(\sim B,W,U,\sim D)$$

$$P(\sim B,W,U,\sim D) = P(U) \cdot P(W|U) \cdot P(\sim B|U) \cdot P(\sim D|W,\sim B)$$

$$P(U) = P(C)P \cdot P(U|C) + P(\sim C) \cdot P(U|\sim C)$$

代入数据就可算得:

$$P(B|W,U,\sim D) \approx 0.2893$$

由于 $a_B = 0.04 < P(B|W,U,\sim D)$ 故转移到下一个状态

$$\pi_2 = (\sim C, W, U, B, \sim D)$$

3. 采样 D:

$$P(D|B,W) = 0.335$$

由于 $a_B = 0.77 > P(D|B,W)$ 故停留在原状态

由于在顺序采样的过程中 D 恒为 false，所以 $P(D|U,W) = 0$

第 2 问. 求解 $P(D|U,W)$

可以使用以下公式：

$$P(D|U,W) = \frac{P(D,U,W)}{P(U,W)}$$

由于证据变量 U，W 固定为 true

$$P(D|U,W) = P(D,U,W) \tag{1}$$

根据吉布斯采样法，我们可以计算 $P(D,U,W)$

根据给定的情况统计信息，我们有：

$$n(D,C,U,W,B) = 25$$

$$n(D,\sim C,U,W,B) = 20$$

$$n(D,C,U,W,\sim B) = 12$$

$$n(D,\sim C,U,W,\sim B) = 7$$

因此，

$$\begin{aligned} n(D,U,W) &= n(D,C,U,W,B) + n(D,\sim C,U,W,B) \\ &\quad + n(D,C,U,W,\sim B) + n(D,\sim C,U,W,\sim B) \\ &= 25 + 20 + 12 + 7 = 64 \end{aligned}$$

$$P(D,U,W) = \frac{n(D,U,W)}{T} = \frac{64}{1000} = 0.064 \tag{2}$$

由式子 (1)、(2) 可推知

$$P(D|U,W) = 0.064$$

第三题

在高斯混合分布的参数估计中，证明M步中的参数更新公式。

解答

在 EM 算法的 M 步中，我们的目标是最大化 Q 函数

$$Q(\theta,\theta^{(i)})=\sum_{k=1}^{K}\left\{n_k\log\alpha_k+\sum_{j=1}^{N}\hat{\gamma}_{jk}\left[\log\left(\frac{1}{\sqrt{2\pi}}\right)-\log\sigma_k-\frac{1}{2\sigma_k^2}(y_j-\mu_k)^2\right]\right\}$$

其中 $n_k=\sum_{j=1}^{N}\hat{\gamma}_{jk}\quad\sum_{k=1}^{K}n_k=N$

1. 更新 μ_k

我们只考虑与 μ_k 相关的项:

$$Q(\theta,\theta^{(i)})=-\sum_{k=1}^{K}\sum_{j=1}^{N}\hat{\gamma}_{jk}\frac{1}{2\sigma_k^2}(y_j-\mu_k)^2$$

对 μ_k 求偏导:

$$\begin{aligned}\frac{\partial Q}{\partial\mu_k}&=\sum_{j=1}^{N}\hat{\gamma}_{jk}\cdot\frac{\partial}{\partial\mu_k}\left(-\frac{1}{2\sigma_k^2}(y_j-\mu_k)^2\right)\\&=\sum_{j=1}^{N}\hat{\gamma}_{jk}\cdot\left(-\frac{1}{2\sigma_k^2}\cdot2(y_j-\mu_k)\cdot(-1)\right)\\&=\sum_{j=1}^{N}\hat{\gamma}_{jk}\cdot\frac{y_j-\mu_k}{\sigma_k^2}\\&\overset{let}{=}0\\&\Rightarrow\sum_{j=1}^{N}\hat{\gamma}_{jk}y_j=\mu_k\sum_{j=1}^{N}\hat{\gamma}_{jk}\\&\Rightarrow\hat{\mu}_k=\frac{\sum_{j=1}^{N}\hat{\gamma}_{jk}y_j}{\sum_{j=1}^{N}\hat{\gamma}_{jk}}\end{aligned}$$

2. 更新 σ_k^2

现在，对 Q 函数中与 σ_k^2 相关的部分进行求导：

$$Q(\theta, \theta^{(i)}) = \sum_{k=1}^{K} \sum_{j=1}^{N} \hat{\gamma}_{jk} \left(-\log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right)$$

对 σ_k^2 求偏导：

$$\begin{aligned}
\frac{\partial Q}{\partial \sigma_k^2} &= \sum_{j=1}^{N} \hat{\gamma}_{jk} \cdot \frac{\partial}{\partial \sigma_k^2} \left(-\log \sigma_k - \frac{1}{2\sigma_k^2} (y_j - \mu_k)^2 \right) \\
&= \sum_{j=1}^{N} \hat{\gamma}_{jk} \left(-\frac{1}{2\sigma_k^2} + \frac{(y_j - \mu_k)^2}{2\sigma_k^4} \right) \\
&\overset{let}{=} 0 \\
&\Rightarrow \hat{\sigma}_k^2 = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} (y_j - \mu_k)^2}{\sum_{j=1}^{N} \hat{\gamma}_{jk}}
\end{aligned}$$

3. 更新 α_k

最后，对 Q 函数中与 α_k 相关的部分进行求偏导：

$$Q(\theta, \theta^{(i)}) = \sum_{k=1}^{K} n_k \log \alpha_k$$

由于 α_k 是概率，需要满足约束条件 $\sum_{k=1}^{K} \alpha_k = 1$。引入拉格朗日乘子 λ，定义拉格朗日函数：

$$\mathcal{L} = \sum_{k=1}^{K} n_k \log \alpha_k + \lambda \left(1 - \sum_{k=1}^{K} \alpha_k \right)$$

对 α_k 求导并令其为零：

$$\frac{\partial \mathcal{L}}{\partial \alpha_k} = \frac{n_k}{\alpha_k} - \lambda = 0$$

得到：

$$\alpha_k = \frac{n_k}{\lambda}$$

结合约束条件 $\sum_{k=1}^{K} \alpha_k = 1$：

$$\sum_{k=1}^{K} \frac{n_k}{\lambda} = 1 \Rightarrow \lambda = \sum_{k=1}^{K} n_k = N$$

因此，α_k 的更新公式为：

$$\hat{\alpha}_k = \frac{n_k}{N} = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk}}{N}$$

综上

$$\begin{aligned}
\hat{\mu}_k &= \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} y_j}{\sum_{j=1}^{N} \hat{\gamma}_{jk}} && k = 1, 2, ..., K \\
\hat{\sigma}_k^2 &= \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} (y_j - \mu_k)^2}{\sum_{j=1}^{N} \hat{\gamma}_{jk}} && k = 1, 2, ..., K \\
\hat{\alpha}_k &= \frac{n_k}{N} = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk}}{N} && k = 1, 2, ..., K
\end{aligned}$$

第四题

已知观测数据 -67, -48, 6, 8, 14, 16, 23, 24, 28, 29, 41, 49, 56, 60, 75，试用EM模型估计两个分量的高斯混合模型的5个参数。

解答

解答思路：

两个分量的高斯混合模型一共有 6 个参数 $\mu_1, \mu_2, \delta_1, \delta_2, \alpha_1, \alpha_2$，其中 α_2 可由 $\alpha_2 = 1 - \alpha_1$ 得到，故仅估计 5 个参数即可。

1. 写出高斯混合模型；
2. 写出高斯混合模型参数估计的 EM 算法；
3. 采用 sklearn 的 GaussianMixture 计算 6 个参数；
4. 采用自编程实现高斯混合模型的 EM 算法。

解答步骤：

第 1 步：高斯混合模型

高斯混合模型是指具有如下形式的概率分布的模型：

$$P(y|\theta) = \sum_{k=1}^{K} \alpha_k \varphi(y|\theta_k)$$

其中，α_k 是系数，$\alpha_k \geq 0, \sum_{k=1}^{K} \alpha_k = 1$；

$\varphi(y|\theta_k)$ 是高斯分布密度，$\theta_k = (u_k, \sigma_k^2)$，

$$\varphi(y|\theta_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(y-\mu_k)^2}{2\sigma_k^2}\right)$$

称为第 k 个分模型。

从上述描述中可知，如果是 2 个高斯混合分模型，一共需要估计的参数有 6 个 $\mu_1, \mu_2, \sigma_1, \sigma_2, \alpha_1, \alpha_2$，其中 $\alpha_1 + \alpha_2 = 1$。

第 2 步：高斯混合模型参数估计的 EM 算法

输入：观测数据 $y_1, y_2, \ldots, y_N$，高斯混合模型；

输出：高斯混合模型参数的估计。

(1) 取参数的初始值开始迭代；

(2) E 步：依据当前模型参数，计算每个分模型对观测数据 y_j 的响应度

$$\hat{\gamma}_{jk} = \frac{\alpha_k \varphi(y_j|\theta_k)}{\sum_{k=1}^{K} \alpha_k \varphi(y_j|\theta_k)}, \quad j = 1, 2, \ldots, N; \quad k = 1, 2, \ldots, K$$

(3) M 步：计算新一轮迭代的模型参数

$$\hat{\mu}_k = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} y_j}{\sum_{j=1}^{N} \hat{\gamma}_{jk}}, \quad k = 1, 2, \ldots, K$$

$$\hat{\sigma}_k^2 = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} (y_j - \mu_k)^2}{\sum_{j=1}^{N} \hat{\gamma}_{jk}}, \quad k = 1, 2, \ldots, K$$

$$\hat{\alpha}_k = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk}}{N}, \quad k = 1, 2, \ldots, K$$

(4) 重复第 (2) 步和第 (3) 步，直到收敛。

第 3 步：采用 sklearn 的 GaussianMixture 计算 6 个参数

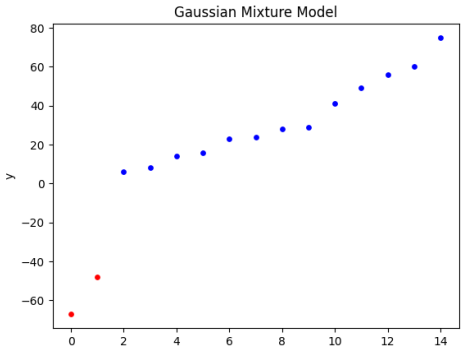
可在附录查看完整代码，请参考 GaussianMixture 模型。

模型运行结果如下，分别计算出参数结果以及描述数据点属于哪一个高斯分布，同时，使用 sklearn 的 GaussianMixture 无需手动赋初值，模型会自动计算出合适的初值。

```
分类结果: labels = [0 0 1 1 1 1 1 1 1 1 1 1 1 1]

两个分量高斯混合模型的6个参数如下:
weights =  [[0.13317228 0.86682772]]
means = [[-57.51107674  32.98488749]]
stds = [[ 90.24987868 429.45833107]]
```

(a) 参数计算结果



(b) 聚类结果可视化

图 2: 运行结果

第 4 步：自编程实现高斯混合模型的 EM 算法

可在附录查看完整代码，请参考 自编码 GMM。

该代码实现了一个高斯混合模型类，用于估计观测数据的分布。类初始化时设置模型参数的初值，fit 方法通过 EM 算法迭代更新模型参数，直到收敛条件满足。同时采用网格搜索法，通过不同初始值寻找局部最优解，最终输出最佳模型参数。模型运行结果如下：

```
weights : [[0.56950675 0.43049325]]
means : [[27.41762854 12.35515017]]
stds : [[ 268.17311145 2772.33989897]]
```

图 3: 运行结果

可以观察到，自编码计算出的参数与 sklearn 计算出的参数不一致，这是由于初始化参数的策略不一致导致的，使用更合理的初始化参数策略可以计算出逼近 sklearn 的参数值。

第五题

考虑一个伯努利混合分布，即

$$p(x;\mu,\pi)=\sum_{k=1}^{K}\pi_k p(x;\mu_k),$$

其中 $p(x;\mu_k)=\mu_k^x(1-\mu_k)^{(1-x)}$ 为伯努利分布.

给定一组训练集合 $D=\{x^{(1)},x^{(2)},\cdots,x^{(N)}\}$，若用 EM 算法来进行参数估计，推导其每步的参数更新公式.

解答

1. 明确隐变量，写出完全数据的对数似然函数

可以设想观测数据 $x^{(j)},j=1,2,...,N$ 是这样产生的: 首先依概率 π_k 选择第 k 个伯努利分布模型 $P(x|\mu_k)$, 然后依第 k 个分模型的概率分布 $P(x|\mu_k)$ 生成观测数据 $x^{(j)}$。

这时观测数据 $x^{(j)},j=1,2,...,N$ 是已知的; 反映观测数据来自第 k 个分模型的数据是未知的, k = 1, 2 ,...,K 以隐变量 γ_{jk} 表示, 定义如下:

$$\gamma_{jk}=\begin{cases}1 & \text{第 j 个观测来自于第 k 个模型}\\ 0 & \text{其他}\end{cases}$$

$$j=1,2,...,N;k=1,2,...,K$$

γ_{jk} 是 0-1 随机变量

有了观测数据 $x^{(j)}$ 和未观测数据 γ_{jk}，那么完全数据是

$$(x^{(j)},\gamma_{j1},\gamma_{j2},...,\gamma_{jK}),j=1,2,...,N.$$

于是，可以得到完全数据的似然函数

$$\begin{aligned}
P(x,\gamma|\mu) &= \prod_{j=1}^{N} P(x^{(j)},\gamma_{j1},\gamma_{j2},...,\gamma_{jK}) \\
&= \prod_{k=1}^{K}\prod_{j=1}^{N}[\pi_k P(x^{(j)}|\mu_k)]^{\gamma_{jk}} \\
&= \prod_{k=1}^{K}\pi_k^{n_k}\prod_{j=1}^{N}[P(x^{(j)}|\mu_k)]^{\gamma_{jk}} \\
&= \prod_{k=1}^{K}\pi_k^{n_k}\prod_{j=1}^{N}\{\mu_k^{x^{(j)}}(1-\mu_k)^{1-x^{(j)}}\}^{\gamma_{jk}}
\end{aligned}$$

式中 $n_k = \sum_{j=1}^{N}\gamma_{jk} \quad \sum_{k=1}^{K} n_k = N$

完全数据的对数似然函数为

$$logP(x,\gamma|\mu) = \sum_{k=1}^{K}\left\{n_k log\pi_k + \sum_{j=1}^{N}\gamma_{jk}[x^{(j)}log\mu_k + (1-x^{(j)})log(1-\mu_k)]\right\}$$

2. E 步

确定 Q 函数

$$Q(\mu,\mu^{(i)}) = \sum_{k=1}^{K}\left\{n_k log\pi_k + \sum_{j=1}^{N}\hat{\gamma}_{jk}[x^{(j)}log\mu_k + (1-x^{(j)})log(1-\mu_k)]\right\}$$

其中,$\hat{\gamma}_{jk}$ 是在当前模型下第 j 个观测数据来自于第 k 个分模型的概率，称为分模型 k 对观测数据 $x^{(j)}$ 的响应度。

计算响应度期望

$$\begin{aligned}
\hat{\gamma}_{jk} &= \mathbb{E}(\gamma_{jk}|x^{(j)},\mu^{(i)}) \\
&= P(\gamma_{jk}=1|x^{(j)},\mu^{(i)})\times 1 + P(\gamma_{jk}=0|x^{(j)},\mu^{(i)})\times 0 \\
&= P(\gamma_{jk}=1|x^{(j)},\mu^{(i)}) \\
&= \frac{P(\gamma_{jk}=1|\mu^{(i)})P(x^{(j)}|\gamma_{jk}=1,\mu^{(i)})}{\sum_{k=1}^{K}P(\gamma_{jk}=1|\mu^{(i)})P(x^{(j)}|\gamma_{jk}=1,\mu^{(i)})} \quad \text{(贝叶斯公式)} \\
&= \frac{\pi_k P(x^{(j)}|\mu^{(i)})}{\sum_{k=1}^{K}\pi_k P(x^{(j)}|\mu^{(i)})}
\end{aligned}$$

最后得到 $\hat{\gamma}_{jk}$

$$\hat{\gamma}_{jk} = \frac{\pi_k P(x^{(j)}|\mu^{(i)})}{\sum_{k=1}^{K} \pi_k P(x^{(j)}|\mu^{(i)})} j = 1, 2, ..N; k = 1, 2, ..., K.$$

3. M 步

迭代的 M 步是就函数 $Q(\mu, \mu^{(i)})$ 对 μ 的极大值，即求新一轮迭代的模型参数。Q 函数在 E 步中已求出，表达式如下

$$Q(\mu, \mu^{(i)}) = \sum_{k=1}^{K} \left\{ n_k log\pi_k + \sum_{j=1}^{N} \hat{\gamma}_{jk} [x^{(j)} log\mu_k + (1 - x^{(j)}) log(1 - \mu_k)] \right\}$$

更新 μ_k

对 μ_k 进行求偏导并令其等于零：

$$\frac{\partial Q}{\partial \mu_k} = \sum_{j=1}^{N} \hat{\gamma}_{jk} \left(\frac{x^{(j)}}{\mu_k} - \frac{1 - x^{(j)}}{1 - \mu_k} \right) = 0$$

最后解得

$$\hat{\mu}_k = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} x^{(j)}}{\sum_{j=1}^{N} \hat{\gamma}_{jk}}$$

更新 π_k

我们需要满足约束条件：

$$\sum_{k=1}^{K} \pi_k = 1$$

为了解决这个约束问题，我们使用拉格朗日乘子法。定义拉格朗日函数：

$$\mathcal{L}(\pi_1, \pi_2, \ldots, \pi_K, \lambda) = Q(\mu, \mu^{(i)}) - \lambda \left(\sum_{k=1}^{K} \pi_k - 1 \right)$$

对拉格朗日函数 $\mathcal{L}$ 关于 π_k 和 λ 求导，并令其等于零：

对 π_k 的偏导数：

$$\frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{n_k}{\pi_k} - \lambda = 0$$

即

$$\pi_k = \frac{n_k}{\lambda}$$

将 π_k 代入约束条件中，得到：

$$\sum_{k=1}^{K} \frac{n_k}{\lambda} = 1$$

从而可以得到：

$$\lambda = \sum_{k=1}^{K} n_k = N$$

代入上式可得

$$\hat{\pi}_k = \frac{n_k}{N} = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk}}{N}$$

最终更新公式

- 更新成功概率 μ_k:

$$\hat{\mu}_k = \frac{\sum_{j=1}^{N} \hat{\gamma}_{jk} x^{(j)}}{\sum_{j=1}^{N} \hat{\gamma}_{jk}}$$

- 更新权重 π_k:

$$\hat{\pi}_k = \frac{1}{N} \sum_{j=1}^{N} \hat{\gamma}_{jk}$$

4. 重复第 2 步和第 3 步，直至收敛

参考文献

[1] https://datawhalechina.github.io/statistical-learning-method-solutions-manua

[2] https://www.bilibili.com/video/BV1TM4y1m7BN/?vd_source=f90de220471b49ff28d579f90952801a

[3] https://wenku.baidu.com/view/d32bdd3a1b37f111f18583d049649b6648d7099a.html?_wkts_=1730706100734

[4] 李航. 统计学习方法.

A 三硬币模型

```python
import math

class ThreeCoinEM:
    def __init__(self, prob, tol=1e-6, max_iter=1000):
        """
        初始化模型参数
        :param prob: 模型参数的初值
        :param tol: 收敛阈值
        :param max_iter: 最大迭代次数
        """
        self.prob_A, self.prob_B, self.prob_C = prob
        self.tol = tol
        self.max_iter = max_iter

    def calc_mu(self, j):
        """
        （E步）计算mu
        :param j: 观测数据y的第j个
        :return: 在模型参数下观测数据yj来自掷硬币B的概率
        """
        # 掷硬币A观测结果为正面
        pro_1 = self.prob_A * \
            math.pow(self.prob_B, data[j]) * \
            math.pow((1 - self.prob_B), 1 - data[j])
        # 掷硬币A观测结果为反面
        pro_2 = (1 - self.prob_A) * math.pow(self.prob_C,data[j
            ]) * math.pow((1 - self.prob_C), 1 - data[j])
        return pro_1 / (pro_1 + pro_2)

    def fit(self, data):
        count = len(data)
```

```
31          print("模型参数的初值: ")
32          print("prob_A={}, prob_B={}, prob_C={}".format(
33              self.prob_A, self.prob_B, self.prob_C))
34          print("EM算法训练过程: ")
35          for i in range(self.max_iter):
36              # (E步) 得到在模型参数下观测数据yj来自掷硬币B的概率
37              _mu = [self.calc_mu(j) for j in range(count)]
38              # (M步) 计算模型参数的新估计值
39              prob_A = 1 / count * sum(_mu)
40              prob_B = sum([_mu[k] * data[k] for k in range(count)
                    ]) \
41                   / sum([_mu[k] for k in range(count)])
42              prob_C = sum([(1 - _mu[k]) * data[k] for k in range(
                    count)]) \
43                   / sum([(1 - _mu[k]) for k in range(count)])
44              print('第{}次: prob_A={:.4f}, prob_B={:.4f}, prob_C
                    ={:.4f}'.format(
45                   i + 1, prob_A, prob_B, prob_C))
46              # 计算误差值
47              error = abs(self.prob_A - prob_A) + \
48                   abs(self.prob_B - prob_B) + abs(self.prob_C -
                        prob_C)
49              self.prob_A = prob_A
50              self.prob_B = prob_B
51              self.prob_C = prob_C
52              # 判断是否收敛
53              if error < self.tol:
54                  print("模型参数的极大似然估计: ")
55                  print("prob_A={:.4f}, prob_B={:.4f}, prob_C={:.4
                        f}".format(self.prob_A, self.prob_B,self.
                        prob_C))
56                  break
57
```

```
58 if __name__=='__main__':
59     # 加载数据
60     data = [1, 1, 0, 1, 0, 0, 1, 0, 1, 1]
61     # 模型参数的初值
62     init_prob = [0.46, 0.55, 0.67]
63     # 三硬币模型的EM模型
64     em = ThreeCoinEM(prob=init_prob, tol=1e-5, max_iter=100)
65     # 模型训练
66     em.fit(data)
```

B GaussianMixture 模型

```
1  from sklearn.mixture import GaussianMixture
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # 初始化观测数据
6  data = np.array([-67, -48, 6, 8, 14, 16, 23, 24, 28,
7                   29, 41, 49, 56, 60, 75]).reshape(-1, 1)
8
9  # 设置n_components=2, 表示两个分量高斯混合模型
10 gmm_model = GaussianMixture(n_components=2)
11 # 对模型进行参数估计
12 gmm_model.fit(data)
13 # 对数据进行聚类
14 labels = gmm_model.predict(data)
15
16 # 得到分类结果
17 print("分类结果: labels = {}\n".format(labels))
18 print("两个分量高斯混合模型的6个参数如下: ")
19 # 得到参数a1, a2
20 print("weights = ", gmm_model.weights_.reshape(1, -1))
```

```
# 得到参数u1,u2
print("means =", gmm_model.means_.reshape(1, -1))
# 得到参数sigma1, sigma1
print("stds =", gmm_model.covariances_.reshape(1, -1))

# 绘制观测数据的聚类情况
for i in range(0, len(labels)):
    if labels[i] == 0:
        plt.scatter(i, data.take(i), s=15, c='red')
    elif labels[i] == 1:
        plt.scatter(i, data.take(i), s=15, c='blue')
plt.title('Gaussian Mixture Model')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

C 自编码 GMM

```
import numpy as np
import itertools

class MyGMM:
    def __init__(self, alphas_init, means_init, covariances_init
        , tol=1e-6, n_components=2, max_iter=50):
        # (1)设置参数的初始值
        # 分模型权重
        self.alpha_ = np.array(
            alphas_init, dtype="float16").reshape(n_components,
                1)
        # 分模型均值
        self.mean_ = np.array(
```

```
12              means_init, dtype="float16").reshape(n_components,
                    1)
13          # 分模型标准差（方差的平方）
14          self.covariances_ = np.array(
15              covariances_init, dtype="float16").reshape(
                    n_components, 1)
16          # 迭代停止的阈值
17          self.tol = tol
18          # 高斯混合模型分量个数
19          self.K = n_components
20          # 最大迭代次数
21          self.max_iter = max_iter
22          # 观测数据
23          self._y = None
24          # 实际迭代次数
25          self.n_iter_ = 0
26
27      def gaussian(self, mean, convariances):
28          """计算高斯分布概率密度"""
29          return 1 / np.sqrt(2 * np.pi * convariances) * np.exp(
30              -(self._y - mean) ** 2 / (2 * convariances))
31
32      def update_r(self, mean, convariances, alpha):
33          """更新r_jk 分模型k对观测数据yi的响应度"""
34          r_jk = alpha * self.gaussian(mean, convariances)
35          return r_jk / r_jk.sum(axis=0)
36
37      def update_params(self, r):
38          """更新u al si 每个分模型k的均值、权重、方差的平方"""
39          u = self.mean_[-1]
40          _mean = ((r * self._y).sum(axis=1) / r.sum(axis=1)).
                reshape(self.K, 1)
41          _covariances = ((r * (self._y - u) ** 2).sum(axis=1) /
```

```
42                          r.sum(axis=1)).reshape(self.K, 1)
43          _alpha = (r.sum(axis=1) / self._y.size).reshape(self.K,
                1)
44          return _mean, _covariances, _alpha
45
46      def judge_stop(self, mean, covariances, alpha):
47          """中止条件判断"""
48          a = np.linalg.norm(self.mean_ - mean)
49          b = np.linalg.norm(self.covariances_ - covariances)
50          c = np.linalg.norm(self.alpha_ - alpha)
51          return True if np.sqrt(a ** 2 + b ** 2 + c ** 2) < self.
                tol else False
52
53      def fit(self, y):
54          self._y = np.copy(np.array(y))
55          """迭代训练获得预估参数"""
56          # (2)E步：计算分模型k对观测数据yi的响应度
57          r = self.update_r(self.mean_, self.covariances_, self.
                alpha_)
58          # 更新r_jk 分模型k对观测数据yi的响应度
59          _mean, _covariances, _alpha = self.update_params(r)
60          # 更新u al si 每个分模型k的均值、权重、方差的平方
61          for i in range(self.max_iter):
62              if not self.judge_stop(_mean, _covariances, _alpha):
63                  # (4)未达到阈值条件，重复迭代
64                  r = self.update_r(_mean, _covariances, _alpha)
65                  # (3)M步：计算新一轮迭代的模型参数
66                  _mean, _covariances, _alpha = self.update_params
                        (r)
67              else:
68                  # 达到阈值条件，停止迭代
69                  self.n_iter_ = i
70                  break
```

```
71
72              self.mean_ = _mean
73              self.covariances_ = _covariances
74              self.alpha_ = _alpha
75
76      def score(self):
77          """计算该局部最优解的score, 即似然函数值"""
78          return (self.alpha_ * self.gaussian(self.mean_, self.
                covariances_)).sum()
79
80  def main():
81      # 观测数据
82      y = np.array([-67, -48, 6, 8, 14, 16, 23, 24, 28,
83                    29, 41, 49, 56, 60, 75]).reshape(1, 15)
84      # 预估均值和方差, 以其邻域划分寻优范围
85      y_mean = y.mean() // 1
86      y_std = (y.std() ** 2) // 1
87
88      # 网格搜索, 对不同的初值进行参数估计
89      alpha = [[i, 1 - i] for i in np.linspace(0.1, 0.9, 9)]
90      mean = [[y_mean + i, y_mean + j]
91              for i in range(-10, 10, 5) for j in range(-10, 10,
                    5)]
92      covariances = [[y_std + i, y_std + j]
93                     for i in range(-1000, 1000, 500) for j in range
                         (-1000, 1000, 500)]
94      results = []
95      for i in itertools.product(alpha, mean, covariances):
96          init_alpha = i[0]
97          init_mean = i[1]
98          init_covariances = i[2]
99          clf = MyGMM(alphas_init=init_alpha, means_init=init_mean
                , covariances_init=init_covariances,
```

```
100                     n_components=2, tol=1e-6)
101         clf.fit(y)
102         # 得到不同初值收敛的局部最优解
103         results.append([clf.alpha_, clf.mean_, clf.covariances_,
                clf.score()])
104     # 根据score, 从所有局部最优解找到相对最优解
105     best_value = max(results, key=lambda x: x[3])
106
107     print("weights : {}".format(best_value[0].T))
108     print("means : {}".format(best_value[1].T))
109     print("stds : {}".format(best_value[2].T))
110
111 if __name__=='__main__':
112     main()
```